

Lecture 4

ESSLLI Course — *Introduction to Homotopy Type Theory / Univalent Foundations*

Tom de Jong

Additional reading

The citations refer to the file `Resources.pdf`. Related material can be found in [Rij25, Sections 9–11, 13, 17]. See [Esc19, Sections 3.2, 3.3, 3.7, 3.8, 3.10–3.12, 3.18, 3.21] for a somewhat comparable treatment in Agda.

1. A stratification of types

A perhaps surprisingly fundamental notion is Voevodsky’s notion of a **contractible** (a.k.a. singleton) type.

Definition. A type A is **contractible** if $\sum(a : A), \prod(b : A), a = b$.

Note that this says that A has exactly one (designated) point.

Example. For any type A with a point $a_0 : A$, the type $\sum(x : A), a_0 = x$ is contractible.

As the designated point we take (a_0, refl) and we have $(a, p) = (a_0, \text{refl})$ by path induction.

This example very useful as will soon see.

It is not too hard ([exercise!](#)) to prove the following, which we shall use often.

Lemma. A type is contractible if and only if it is equivalent to $\mathbf{1}$. ■

Voevodsky also realized that the contractible types sit at the bottom of an infinite hierarchy. (We count from -2 to follow the convention in homotopy theory.)

Definition. For $n \geq -2$, define what it means for A to be an n -type (a.k.a. n -truncated, Voevodsky: “of hlevel $n + 2$ ”):

$$\begin{aligned} \text{is-}(-2)\text{-type}(A) &:= \text{is-contr}(A) && (\equiv \sum(a : A), \prod(b : A), a = b) \\ \text{is-}(-1)\text{-type}(A) &:= \text{is-prop}(A) && (\equiv \prod(a, b : A), a = b) \\ \text{is-}0\text{-type}(A) &:= \prod(a, b : A), \text{is-prop}(a = b) \\ \text{is-}(n+1)\text{-type}(A) &:= \prod(a, b : A), \text{is-}n\text{-type}(a = b) \end{aligned}$$

It is equivalent (with some work! cf. [Corollary\[proposition-iff-contractible-id\]](#)) and looks prettier, but is less convenient, to adopt the last clause even for $n = -2$, i.e. to define

$\text{is-}(-1)\text{-type}(A) := \prod(a, b : A), \text{is-contr}(a = b)$.

We said that this is a hierarchy and indeed we can show that $\text{is-}n\text{-type}(A)$ implies $\text{is-}(n+1)\text{-type}(A)$, see below.

Sets and propositions

Note that $\text{is-}0\text{-type}(A)$ says that the identifications of A are not interesting: any two are equal. We also call 0-types **sets**, because in our intuitive conception of ordinary sets like $\mathbb{N}$ they have no interesting identifications. (We will prove this formally below.)

For much “ordinary” mathematics, we could only do with propositions and sets, but there are **genuine examples of higher types**, such as type universes (after univalence; this lecture) and the circle (next lecture).

This is also a good time to revisit our monoid example. We previously said that the type of monoids was a type together with a multiplication and a neutral elements, satisfying some equations. But a faithful

encoding of the notion of a monoid requires the type to be a set. Why? Because this ensures that for instance the type

$$\prod (x, y, z : M), (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

expressing associativity of the multiplication is a proposition and therefore a property and **not** an additional *piece of data*.

We could consider having this as data, but then presumably homomorphisms should preserve this data (and maybe extra data too!) and this would be a whole new theory, somewhat similar to **but not the same as** the theory of monoids that we're used to.

Restricting to sets, we can also use the usual argument from algebra that the neutral element is unique in a monoid to prove that the type $\sum (e : M), (\prod (x : M), e \cdot x = x) \times (\prod (x : M), x \cdot e = x)$ is a proposition, so that this type is equivalent to its propositional truncation.

Hence, it does not really matter if we formulate the type of monoids with the neutral element via logical existence or as an additional piece of data.

The hierarchy of n -types

Definition. A map $f : A \rightarrow B$ is **(weakly) constant** if $\prod (a, a' : A), f(a) = f(a')$.

Lemma (Hedberg). If, for every $x : A$, we have a (weakly) constant endomap on $a = x$, then $a = x$ is a proposition for every $x : A$.

Proof. For $x : A$, let f_x be the constant endomap on $a = x$. By path induction it follows that we have

$$\prod (x : X), \prod (p : a = x), p = (f_a(\text{refl}))^{-1} \bullet f_x(p).$$

Hence, given $p, q : a = x$, we have

$$p = (f_a(\text{refl}))^{-1} \bullet f_x(p) \xrightarrow{\text{as } f_x \text{ is constant}} (f_a(\text{refl}))^{-1} \bullet f_x(q) = q,$$

so $p = q$ as desired. ■

Lemma. If A is **decidable** (i.e. $A + \neg A$), then it has a constant endomap.

Proof. Given $\delta : A + \neg A$, we have $\delta \equiv \text{inl}(a_0)$ or $\delta \equiv \text{inr}(\nu)$. In the first case, we simply take $a \mapsto a_0$ as a constant endomap on A .

In the second case, we can take the identity on A . This map will be constant, because, given $a, b : A$, to prove $a = b$, it suffices to prove $\emptyset$, which we can using ν and a (or b). ■

Corollary. Any type with decidable equality (i.e. $x = y$ is decidable for all x and y) must be a set.

Proof. By Hedberg's lemma and the previous lemma. ■

Corollary. Any proposition is a set.

Proof. Any proposition trivially has decidable equality: we always have $x = y$ in a proposition.

Corollary. Any n -type is also an $(n + 1)$ -type.

Proof. By induction on n , it suffices to prove that contractible types are propositions and that propositions are sets. The former is straightforward and the latter was just shown. ■

Corollary[proposition-iff-contractible-id] A type is a proposition if and only if all its identity types are contractible.

Proof. If all its identity types are contractible, then in particular we have an element of each of them, so that the type is a proposition.

Conversely, suppose that P is a proposition, and let $p, q : P$. Since P is a proposition, we have $i : p = q$ and it suffices to prove that $p = q$ is a proposition, but this follows since propositions are sets. ■

Lemma[is-prop-simplified] If we want to prove $\text{is-prop}(P)$, then we may assume P , i.e. $(P \rightarrow \text{is-prop}(P)) \rightarrow \text{is-prop}(P)$.

Proof. Given $h : P \rightarrow \text{is-prop}(P)$, let $p, q : P$, and we must show $p = q$. Now simply use $h(p)(p)(q)$. ■

Corollary. The type $\text{is-}n\text{-type}(A)$ is a proposition. In particular, being contractible is a property.

Proof. By induction on n , it suffices to show that $\text{is-contr}(A)$ and $\text{is-prop}(A)$ are propositions. The latter follows from Lemma[is-prop-simplified] and the fact that propositions are sets. For the former, we use Lemma[is-prop-simplified] again to assume that A is contractible, and in particular a proposition. Hence, given $(a, p), (a', p') : \text{is-contr}(A)$, we have that $a = a'$ as A is a proposition and $p = p'$ follows (via function extensionality) from the fact that $a = x$ is a proposition for all $x : A$, as propositions are sets. ■

2. Characterizing identity types

Products

We claim that for $(a, b), (a', b') : A \times B$, we have an equivalence $(a, b) = (a', b') \simeq (a = a') \times (b = b')$.

Indeed, we have maps back and forth given by path induction

$$\begin{aligned} (a = b) &= (a', b') \longleftrightarrow (a = a') \times (b = b') \\ \text{refl} &\mapsto (\text{refl}, \text{refl}) \\ \text{refl} &\leftarrow (\text{refl}, \text{refl}) \end{aligned}$$

that moreover compose to the identity (by path induction again).

Dependent pairs

Suppose we have $(a, b) : \sum(x : A), B(x)$. Given another dependent pair $(a', b') : \sum(x : A), B(x)$, one might be tempted to say that $(a, b) = (a', b')$ should be equivalent to $(a = a') \times (b = b')$, but note that this does not typecheck. Indeed, $b : B(a)$, while $b' : B(a')$, so we can't even form the type $b = b'$ unless $a \equiv a'$. However, we can *first* consider $p : a = a'$, and then compare $\text{transport}^B(b, p) : B(a')$ with b' .

Similarly to the above proof, we can then show by path induction that

$$(a, b) = (a', b') \simeq \sum(p : a = a'), \text{transport}^B(b, p) = b'.$$

Subtypes

Consider a subtype $\sum(x : A), P(x)$ of A and two elements (a, p) and (a', p') of it. The above tells us that $(a, p) = (a', p')$ is equivalent to $\sum(e : a = a'), \text{transport}^P(p, e) = p'$. The second component is an identity type of $P(a')$. Since $P(x)$ is a proposition for every $x : A$ (by def. of a subtype), this identity type is contractible, so that:

$$\begin{aligned} (a, p) = (a', p') &\simeq \sum(e : a = a'), \text{transport}^P(p, e) \overline{\overline{P(a')}} p' \\ &\simeq (a = a') \times \mathbb{1} \\ &\simeq a = a'. \end{aligned}$$

That is, the identifications of the subtype are fully characterized by identifications of A .

Example: We will soon show that $\text{is-equiv}(f)$ is a proposition. Hence, two equivalences $f, g : A \simeq B$ are identified precisely when they are identified as (ordinary) *functions* $f, g : A \rightarrow B$. The witnesses that f and g are equivalences need not be compared; they are automatically identified.

Function types

We previously postulated function extensionality as requiring a map $\prod(a : A) f(a) = g(a) \longrightarrow f = g$.

In line with the above equivalences, we would actually like these types to be equivalent. Note that by path induction, there is always a map $f = g \longrightarrow \prod(a : A), f(a) = g(a)$, which sends $\text{refl}(f)$ to $\lambda a. \text{refl}(f(a))$, and (following Voevodsky) we **ask that this map is an equivalence**.

As a consequence we get a map in other direction, recovering our previous formulation.

This “strong” formulation of function extensionality is actually equivalent to our previous formulation, but this is nontrivial. (See [Esc19, Section 3.18].) We will just assume the stronger formulation.

Universes and the univalence axiom

Given $A, B : \mathcal{U}$, we can consider $A \stackrel{\overline{\mathcal{U}}}{=} B$, which raises the question: when should we expect two types to be identified?

Plain MLTT does not answer this question, but Voevodsky’s brilliant answer is to adopt the **univalence axiom** which roughly says that types are equal when they are equivalent.

We write $A \simeq B$ for the **type of equivalences** from A to B .

More precisely, a universe $\mathcal{U}$ is said to be **univalent** if the canonical map (given by path induction via $\text{refl} \mapsto \text{id}$)

$$\text{idtoeq}_{A,B} : A \stackrel{\overline{\mathcal{U}}}{=} B \longrightarrow A \simeq B$$

is itself an equivalence.

In particular, we get an inverse $\text{eqtoId}_{A,B} : A \simeq B \longrightarrow A \stackrel{\overline{\mathcal{U}}}{=} B$.

Note: It is crucial that we adopted the correct notion of equivalence here. If we had said that an equivalence is simply a map with an inverse, then the univalence axiom would be *provably false*! See Exercise 4.6 of the HoTT Book, or a [solution formalized in Rocq by Mike Shulman](#).

Univalence immediately gives us an example of **a type that is not a set**, namely $\mathcal{U}_0$. This is because $2 : \mathcal{U}_0$ and univalence (of $\mathcal{U}_0$) gives $(2 = 2) \simeq (2 \simeq 2)$ and it not hard to show that the type $2 \simeq 2$ has exactly two elements, namely the identity and the map that swaps 0 and 1. Hence, $(2 = 2) \simeq 2$ and 2 is not a proposition; since propositions are closed under equivalence (i.e. if $X \simeq Y$ and X is a proposition, then so is Y), the type $2 = 2$ is not a proposition either.

Another example is the type Monoid of monoids. With univalence (and some work), one can show that $M \stackrel{\overline{\text{Monoid}}}{=} M'$ is equivalent to the type of monoid isomorphisms from M to M' , and there can be many such isomorphisms, e.g. the identity and the negation map are both monoid isomorphisms on $(\mathbb{Z}, +)$.

3. More on equivalences

Lemma[3-for-2-for-equivalences]. In a commutative triangle (i.e. $\prod(a : A), h(a) = g(f(a))$)

$$\begin{array}{ccc} A & \xrightarrow{h} & C \\ & \searrow f & \nearrow g \\ & B & \end{array}$$

if two of the maps are equivalences, so is the third.

Proof: Direct calculation with inverses. ■

In particular, equivalences are closed under composition which justifies reasoning via chains of equivalences

$$\begin{aligned} X &\simeq Y \\ &\simeq Z \\ &\simeq W \end{aligned}$$

to establish that $X \simeq W$. This is a technique that we will employ frequently.

Some useful equivalences

Lemma[pre-and-post-composition-by-equiv]. If $f : A \rightarrow B$ is an equivalence, then so are

$$(-) \circ f : (B \rightarrow Y) \rightarrow (A \rightarrow Y) \quad \text{and} \quad f \circ (-) : (X \rightarrow A) \rightarrow (X \rightarrow B)$$

for any X and Y .

Proof. The left and right inverses of f directly induce left and right inverses of the pre/post-composition maps. ■

Lemma[Σ -reindex-along-equivalence] Given an equivalence $\varphi : A \simeq A'$ and a dependent type $B(x)$ over A , we have an equivalence

$$\begin{aligned} \sum(x : A), B(x) &\simeq \sum(y : A'), B(\varphi^{-1}(y)) \\ (x, b) &\mapsto (\varphi(x), \text{transport}^B(b, p_y)) \\ (\varphi^{-1}(y), b) &\mapsto (y, b) \end{aligned}$$

where $p_y : \varphi(\varphi^{-1}(y)) = y$.

Proof sketch. The proof that these maps compose to the identity needs the fact that equivalences satisfy an additional coherence (they are “half adjoint equivalences”), see [Esc19, Section 3.16]. For a proof of this fact, see e.g. Theorem 4.2.3 of the HoTT Book or [Rij25, Lemma 10.4.5]. ■

Two lemmas that are quite useful to manipulate types are the following.

Lemma. If $p : a = a'$ and $q : b = b'$, then the types $a = b$ and $a' = b'$ are equivalent.

Proof. Send $r : a = b$ to $p^{-1} \bullet r \bullet q$ with inverse $p \bullet (-) \bullet q^{-1}$. ■

Lemma[Σ -congruence]. If we have equivalences $\varphi_x : A(x) \simeq B(x)$ for all $x : X$, then $\sum(x : X), A(x)$ is equivalent to $\sum(x : X), B(x)$ via $(x, a) \mapsto (x, \varphi_x(a))$.

Proof. Direct calculation with inverses, using the characterization of identity types of Σ -types. ■

Contractible fibers

Voevodsky’s original definition of equivalence was in terms of contractible fibers.

Definition. The **fiber** of $f : A \rightarrow B$ at $b : B$ is the type $\text{fib}_f(b) := \sum(a : A), f(a) = b$.

In HoTT, contractibility captures unique existence (see [Esc19, Section 3.21]), so that we read Voevodsky’s definition as “for every $b : B$, there is a unique $a : A$ with $f(a) = b$.”

Proposition. A map $f : A \rightarrow B$ is an equivalence if and only if all of its fibers are contractible types, i.e. for all $b : B$, the type $\text{fib}_f(b)$ is contractible.

Proof. If f is an equivalence, then we can use Σ -reindex-along-equivalence to prove that

$$\text{fib}_f(b) \equiv \left(\sum(a : A), f(a) = b \right) \simeq \left(\sum(y : B), f(f^{-1}(y)) = b \right) \simeq \left(\sum(y : B), y = b \right) \simeq \mathbb{1}.$$

Conversely, if each $\text{fib}_f(b)$ is contractible with designated point $c_b : \sum(a : A), f(a) = b$, then we may define $g : B \rightarrow A$ by $b \mapsto \text{pr}_1(c_b)$, prove $f(g(b)) = b$ by $\text{pr}_2(c_b)$, and $g(f(a)) = a$ by observing that we have $c_{f(a)} = (a, \text{refl})$ by contractibility, so that $g(f(a)) = \text{pr}_1(c_{f(a)}) = \text{pr}_1(a, \text{refl}) = a$. ■

Since being contractible is a proposition, it follows that Voevodsky’s definition is a proposition. We can now also prove this for the definition of equivalence that we’ve adopted.

Lemma. Being an equivalence is property, i.e. the type $\text{is-equiv}(f)$ is a proposition.

Proof. With the “strong” function extensionality axiom and Σ -congruence, we see that

$$\begin{aligned} \text{is-equiv}(f) &\equiv \left(\sum (g : B \rightarrow A), \prod (b : B), f(g(b)) = b \right) \times \left(\sum (h : B \rightarrow A), \prod (a : A), h(f(a)) = a \right) \\ &\simeq \left(\sum (g : B \rightarrow A), f \circ g = \text{id}_B \right) \times \left(\sum (h : B \rightarrow A), h \circ f = \text{id}_A \right) \\ &\simeq \text{fib}_{f \circ (-)}(\text{id}_B) \times \text{fib}_{(-) \circ f}(\text{id}_A). \end{aligned}$$

Now to prove that $\text{is-prop}(\text{is-equiv}(f))$, we may assume that $\text{is-equiv}(f)$, but then $f \circ (-)$ and $(-) \circ f$ are also equivalences and hence the two fibers above are contractible. ■

Corollary. The types $\text{is-equiv}(f)$ and $\prod (b : B), \text{is-contr}(\text{fib}_f(b))$ are equivalent.

Proof. Both are propositions and we have maps back and forth. ■

A useful consequence of the characterization of equivalences as maps with contractible fibers is the fact that $[\Sigma\text{-congruence}]$ has a converse.

Lemma $[\Sigma\text{-fiberwise-equivalence}]$. Suppose we are given maps $\varphi_x : A(x) \rightarrow B(x)$ for all $x : X$. Then, each φ_x is an equivalence if and only if the induced map $\sum \varphi : \sum (x : X), A(x) \rightarrow \sum (x : X), B(x)$, given by $(x, a) \mapsto (x, \varphi_x(a))$ is.

Proof. Using Σ -reindex-along-equiv we can show that $\text{fib}_{\sum \varphi}(x, b) \simeq \text{fib}_{\varphi_x}(b)$ ([exercise!](#)), so that the lemma follows from the characterization of equivalences as maps with contractible fibers. ■

Example. Finally, we note that the “obvious”/naive definition of $f : A \rightarrow B$ being equivalence as just asking for f to have an inverse is not generally a proposition. Taking f to be the identity on the circle S^1 (next lecture), we can prove using facts from the next lecture that

$$\begin{aligned} &\sum (g : S^1 \rightarrow S^1), \left(\prod (x : S^1), \text{id}(g(x)) = \text{id}(x) \right) \times \left(\prod (x : S^1), \text{id}(g(x)) = \text{id}(x) \right) \\ &\simeq \sum (g : S^1 \rightarrow S^1), (g = \text{id}) \times (g = \text{id}) \\ &\simeq \sum \left(p : \underbrace{\sum (g : S^1 \rightarrow S^1), (g = \text{id})}_{\text{contractible!}} \right) (\text{pr}_1(p) = \text{id}) \\ &\simeq (\text{id} = \text{id}) \\ &\simeq \prod (x : S^1), x = x \\ &\simeq \mathbb{Z}, \end{aligned}$$

making the type very far from contractible.

4. Fundamental theorem of identity types

For characterizing the identity type of products, we could simply define maps back and forth by path induction. For other types, like coproducts, this simple approach does not work. Let’s see why. While we can define $a = a' \rightarrow \text{inl}(a) = \text{inl}(a')$ by path induction as $\text{refl} \mapsto \text{refl}$, we cannot do the same in the reverse direction, because for path induction to be applicable the type would have to be $\text{inl}(a) = x$ for some generic $x : A + B$. But then there is no reason why it should go into $a = a'$; indeed there isn’t even an a' to begin with!

Fortunately there is a general technique to handle situations like these. In the HoTT Book this is introduced as the **encode-decode** method. In later years, different presentations of this technique have

been given, with Egbert Rijke popularizing one based on contractible types under the name **the fundamental theorem of identity types**.

We will state and prove it now, give two basic examples, and use it again in the last lecture for the circle.

Our work on equivalences will also pay off now.

Fundamental theorem of identity types. Suppose we are given a dependent type $C(x)$ over X . Given $x_0 : X$ and $c_0 : C(x_0)$, the maps

$$\begin{aligned} \gamma_x : x_0 = x &\rightarrow C(x) \\ \text{refl} &\mapsto c_0 \quad (\text{i.e. } p \mapsto \text{transport}^C(c_0, p)) \end{aligned}$$

are equivalences for all $x : X$ if and only if $\sum(x : X), C(x)$ is contractible.

Proof. Consider the commutative diagram

$$\begin{array}{ccc} \sum(x : X), x_0 = x & \xrightarrow{\sum \gamma} & \sum(x : X), C(x) \\ & \searrow \simeq & \swarrow \\ & \mathbb{1} & \end{array}$$

By 3-for-2 of equivalences, the right-hand map is an equivalence if and only if $\sum \gamma$ is, which happens if and only if each γ_x is an equivalence by $[\Sigma\text{-fiberwise-equivalence}]$. ■

Example: identity types of coproducts

For coproducts, we show the case where we've fixed $\text{inl}(a_0) : A + B$. Define $C : A + B \rightarrow \mathcal{U}$ by

$$C(\text{inl}(a)) := a = a_0 \quad \text{and} \quad C(\text{inr}(b)) := \mathbb{0}$$

with $c_0 := \text{refl} : C(\text{inl}(a_0))$. We show that the Σ -type of C is contractible:

$$\begin{aligned} \sum(x : A + B), C(x) &\simeq \left(\sum(a : A), C(\text{inl}(a)) \right) + \left(\sum(b : B), C(\text{inr}(b)) \right) \\ &\equiv \left(\sum(a : A), a_0 = a \right) + \left(\sum(b : B), \mathbb{0} \right) \\ &\equiv \left(\sum(a : A), a_0 = a \right) + B \times \mathbb{0} \\ &\simeq \left(\sum(a : A), a_0 = a \right) + \mathbb{0} \\ &\simeq \left(\sum(a : A), a_0 = a \right) \\ &\simeq \mathbb{1}. \end{aligned}$$

This shows that $\text{inl}(a_0) = x \simeq \begin{cases} a_0 = a & \text{if } x \equiv \text{inl}(a) \\ \mathbb{0} & \text{if } x \equiv \text{inr}(b) \end{cases}$

In particular, we see that for $\mathbb{2}$, we have $(0 = 1) \simeq \mathbb{0}$ and $(1 = 1) \simeq (0 = 0) \simeq \mathbb{1}$, so that $\mathbb{2}$ is a set.

Example: identity types of the type of natural numbers

For the type of natural numbers we can use the above to show that $(n = m) = \mathbb{1}$ if $n \equiv m$ and $\mathbb{0}$ else.

It follows that $\mathbb{N}$ is a set.

We define $D : \mathbb{N} \times \mathbb{N} \rightarrow \mathcal{U}_0$ by double induction:

$$\begin{aligned} D(0, 0) &:= \mathbb{1} & D(0, \text{succ}(m)) &:= \mathbb{0}; \\ D(\text{succ}(n), 0) &:= \mathbb{0}; & D(\text{succ}(n), \text{succ}(m)) &:= D(n, m). \end{aligned}$$

We then show that $\sum(y : \mathbb{N}), D(n, y)$ is contractible for every $n : \mathbb{N}$ by induction. For the successor case, we get that $\sum(y : \mathbb{N}), D(n, y)$ is contractible by IH and we calculate

$$\begin{aligned}
& \sum (y : \mathbb{N}), D(\text{succ}(n), y) \\
& \simeq D(\text{succ}(n), 0) + \sum (y : \mathbb{N}), D(\text{succ}(n), \text{succ}(y)) \quad (\text{induced by } \mathbb{N} \simeq \mathbb{1} + \mathbb{N}) \\
& \equiv 0 + \sum (y : \mathbb{N}), D(n, y) \\
& \simeq 0 + \mathbb{1} \quad (\text{by IH}) \\
& \simeq \mathbb{1}.
\end{aligned}$$